DEPSTAR Department of Computer Engineering

Charotar University of Science and Technology

Devang Patel Institute of Advance Technology and Research

Department of Computer Engineering

PRACTICAL-2

PART A

1. Smart thermostat (Thermostat.java, main in this file)
   (a) Create class Thermostat with private String location, private int temperature, private static final int MIN=16 and MAX=30, and private static int activeCount=0.
   (b) Constructor(location, startTemp): store location; set temperature to startTemp if it is within MIN..MAX else 22; increment activeCount.
   (c) Constructor(location): chain with this(location, 22).
   (d) raise(): if temperature < MAX add 1, else print "Already at maximum (30)".
   (e) lower(): if temperature > MIN subtract 1, else print "Already at minimum (16)".
   (f) Add getTemperature() (no setter) and static getActiveCount().
   (g) In main: create two thermostats, call raise() 10 times then lower() 20 times in loops (printing the temperature each time), then print getActiveCount().
   Expected output: temperature rises to 30 then "Already at maximum", falls to 16 then "Already at minimum", count 2.

CODE

```
public class Thermostat {
    private String location;            // where this thermostat is
    private int temperature;            // current temperature (Celsius)

    private static final int MIN = 16;    // shared limits for every thermostat
    private static final int MAX = 30;
    private static int activeCount = 0;   // how many Thermostat objects exist

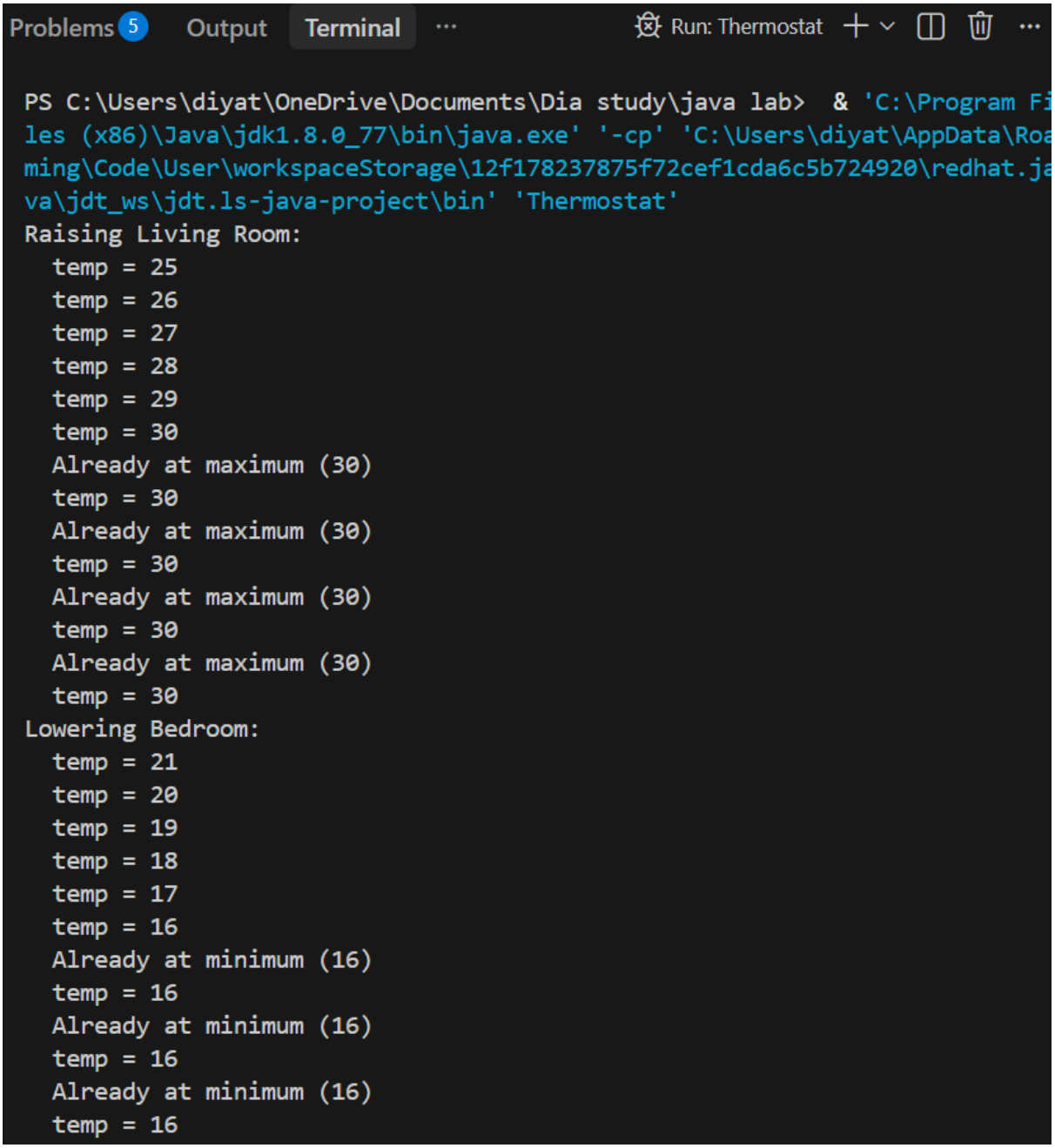
    /** Full constructor: keep startTemp only if it is within the safe range. */
    public Thermostat(String location, int startTemp) {
        this.location = location;
        this.temperature = (startTemp >= MIN && startTemp <= MAX) ? startTemp : 22;
        activeCount++;                  // one more thermostat now exists
    }
    /** Convenience constructor: chains to the full one with a default of 22. */
    public Thermostat(String location) {
        this(location, 22);
    }
```

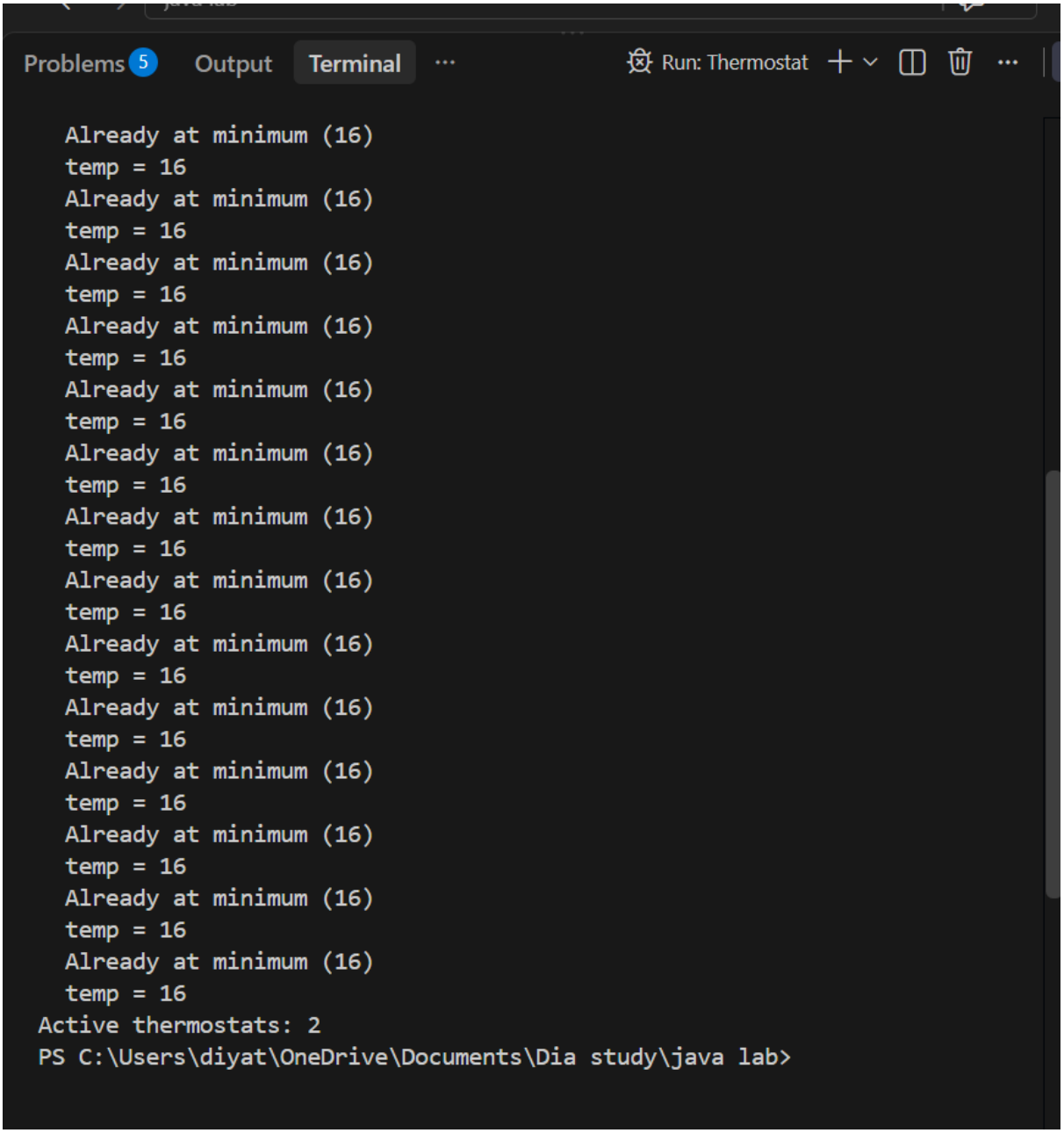
```
    /** Raise by 1, but never above MAX. */
    public void raise() {
        if (temperature < MAX) temperature++;
        else System.out.println("  Already at maximum (" + MAX + ")");
    }
    /** Lower by 1, but never below MIN. */
    public void lower() {
        if (temperature > MIN)
            temperature--;
        else System.out.println("  Already at minimum (" + MIN + ")");
    }
    public int getTemperature() {
        return temperature;
    }   // getter only, NO setter
    public String getLocation() {
        return location;
    }
    public static int getActiveCount() {
         return activeCount;
        }
    public static void main(String[] args) {
        Thermostat living  = new Thermostat("Living Room", 24);
        Thermostat bedroom = new Thermostat("Bedroom");     // defaults to 22

        System.out.println("Raising " + living.getLocation() + ":");
        for (int i = 0; i < 10; i++) {          // try to raise ten times
            living.raise();
            System.out.println("  temp = " + living.getTemperature());
        }

        System.out.println("Lowering " + bedroom.getLocation() + ":");
        for (int i = 0; i < 20; i++) {          // try to lower twenty times
            bedroom.lower();
            System.out.println("  temp = " + bedroom.getTemperature());
        }
        System.out.println("Active thermostats: " + Thermostat.getActiveCount());
    }
}
```

OUTPUT


Problems 5
Output
Terminal
Run: Thermostat
PS C:\Users\diyat\OneDrive\Documents\Dia study\java lab> & 'C:\Program Fi
les (x86)\Java\jdk1.8.0_77\bin\java.exe' '-cp' 'C:\Users\diyat\AppData\Roa
ming\Code\User\workspaceStorage\12f178237875f72cef1cda6c5b724920\redhat.ja
va\jdt_ws\jdt.ls-java-project\bin' 'Thermostat'
Raising Living Room:
temp = 25
temp = 26
temp = 27
temp = 28
temp = 29
temp = 30
Already at maximum (30)
temp = 30
Already at maximum (30)
temp = 30
Already at maximum (30)
temp = 30
Already at maximum (30)
temp = 30
Lowering Bedroom:
temp = 21
temp = 20
temp = 19
temp = 18
temp = 17
temp = 16
Already at minimum (16)
temp = 16
Already at minimum (16)
temp = 16
Already at minimum (16)
temp = 16



CONCLUSION

This practical helped me understand the use of constructors, constructor chaining, static and final variables, and methods in Java. It also demonstrated how to control temperature within a fixed range and count the total number of thermostat objects created.

2. Cinema show (CinemaShow.java, main in this file)

(a) Create class CinemaShow with private String title, private int seatsAvailable, private final int capacity, and private static int totalBooked=0.

(b) Constructor(title, capacity): set both and seatsAvailable = capacity. Constructor(title): chain with this(title, 100).

(c) book(int n): if n ≤ seatsAvailable, reduce seatsAvailable by n, add n to totalBooked, return true; else return false unchanged.

(d) cancel(int n): increase seatsAvailable by n but never above capacity.

(e) Add getSeatsAvailable() and static getTotalBooked().

(f) In main: run a sequence of book and cancel calls, printing the result and seats after each, then print getTotalBooked().

Expected output: booking more than available prints false and leaves seats unchanged; totalBooked equals the successful bookings.

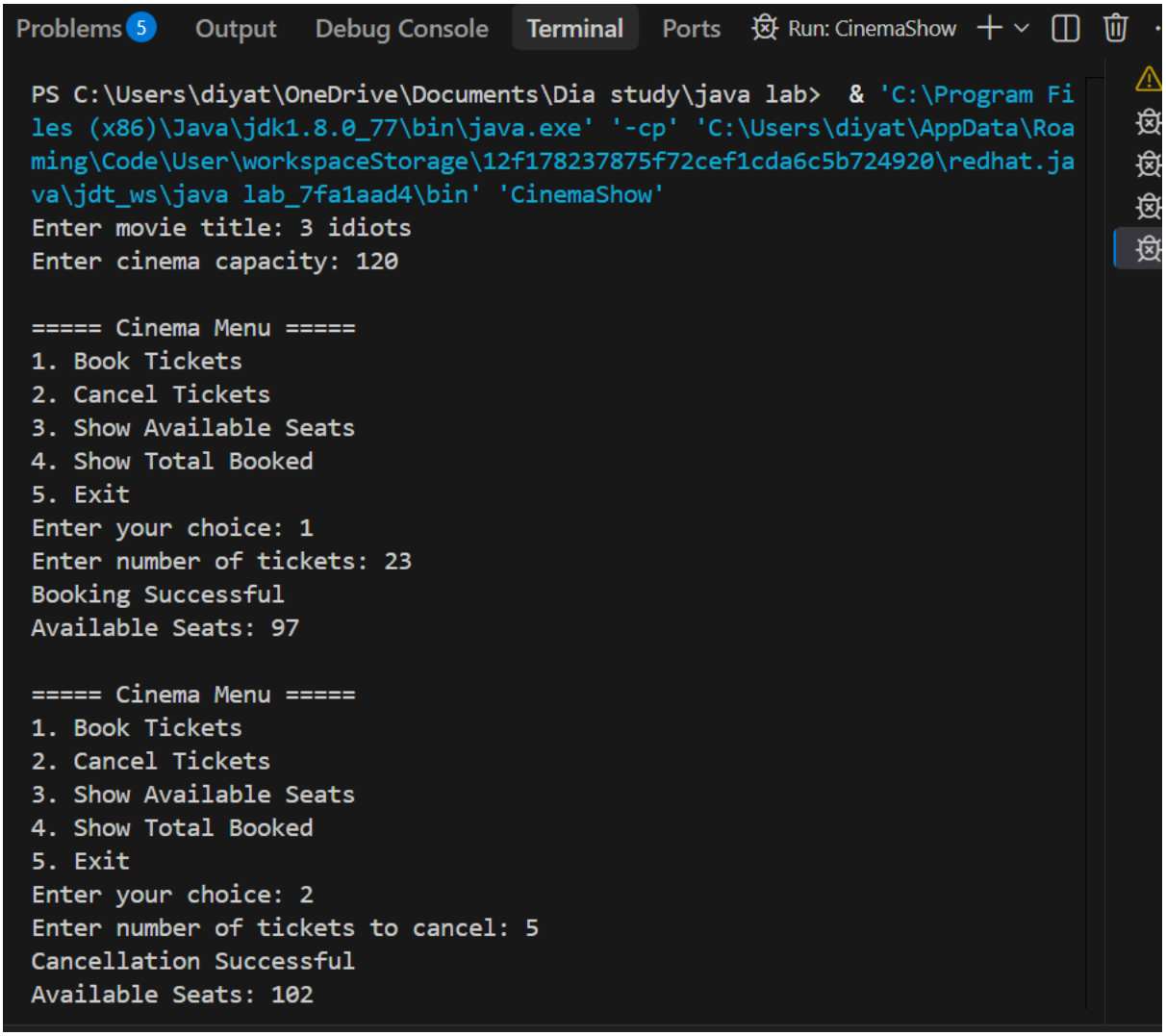
CODE

```
import java.util.Scanner;
class CinemaShow {
    private String title;
    private int seatsAvailable;
    private final int capacity;
    private static int totalBooked = 0;
    CinemaShow(String title, int capacity) {
        this.title = title;
        this.capacity = capacity;
        seatsAvailable = capacity;
    }
    CinemaShow(String title) {
        this(title, 100);
    }
    boolean book(int n) {
        if (n <= seatsAvailable) {
            seatsAvailable = seatsAvailable - n;
            totalBooked = totalBooked + n;
            return true;
        }
        return false;
```

```
    }
    void cancel(int n) {
        seatsAvailable = seatsAvailable + n;
        if (seatsAvailable > capacity) {
            seatsAvailable = capacity;
        }
    }
    int getSeatsAvailable() {
        return seatsAvailable;
    }
    static int getTotalBooked() {
        return totalBooked;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter movie title: ");
        String title = sc.nextLine();
        System.out.print("Enter cinema capacity: ");
        int capacity = sc.nextInt();
        CinemaShow show = new CinemaShow(title, capacity);
        while (true) {
            System.out.println("\n===== Cinema Menu =====");
            System.out.println("1. Book Tickets");
            System.out.println("2. Cancel Tickets");
            System.out.println("3. Show Available Seats");
            System.out.println("4. Show Total Booked");
            System.out.println("5. Exit");
            System.out.print("Enter your choice: ");
            int choice = sc.nextInt();
            if (choice == 1) {
                System.out.print("Enter number of tickets: ");
                int n = sc.nextInt();
                if (show.book(n)) {
                    System.out.println("Booking Successful");
                } else {
                    System.out.println("Booking Failed");
                }
                System.out.println("Available Seats: " + show.getSeatsAvailable());
            }
            else if (choice == 2) {
                System.out.print("Enter number of tickets to cancel: ");
                int n = sc.nextInt();
                show.cancel(n);
                System.out.println("Cancellation Successful");
                System.out.println("Available Seats: " + show.getSeatsAvailable());
            }
            else if (choice == 3) {
                System.out.println("Available Seats: " + show.getSeatsAvailable());
```

```
        }
        else if (choice == 4) {
            System.out.println("Total Booked Tickets: " + CinemaShow.getTotalBooked());
        }
        else if (choice == 5) {
            System.out.println("\nMovie: " + title);
            System.out.println("Available Seats: " + show.getSeatsAvailable());
            System.out.println("Total Booked Tickets: " + CinemaShow.getTotalBooked());
            System.out.println("Thank You!");
            break;
        }
        else {
            System.out.println("Invalid Choice.");
        }
    }
    sc.close();
  }
}
```

OUTPUT


Problems 5
Output
Debug Console
Terminal
Ports
Run: CinemaShow
PS C:\Users\diyat\OneDrive\Documents\Dia study\java lab> & 'C:\Program Files (x86)\Java\jdk1.8.0_77\bin\java.exe' '-cp' 'C:\Users\diyat\AppData\Roaming\Code\User\workspaceStorage\12f178237875f72cef1cda6c5b724920\redhat.java\jdt_ws\java lab_7fa1aad4\bin' 'CinemaShow'
Enter movie title: 3 idiots
Enter cinema capacity: 120
===== Cinema Menu =====
1. Book Tickets
2. Cancel Tickets
3. Show Available Seats
4. Show Total Booked
5. Exit
Enter your choice: 1
Enter number of tickets: 23
Booking Successful
Available Seats: 97
===== Cinema Menu =====
1. Book Tickets
2. Cancel Tickets
3. Show Available Seats
4. Show Total Booked
5. Exit
Enter your choice: 2
Enter number of tickets to cancel: 5
Cancellation Successful
Available Seats: 102

```
===== Cinema Menu =====
1. Book Tickets
2. Cancel Tickets
3. Show Available Seats
4. Show Total Booked
5. Exit
Enter your choice: 3
Available Seats: 102

===== Cinema Menu =====
1. Book Tickets
2. Cancel Tickets
3. Show Available Seats
4. Show Total Booked
5. Exit
Enter your choice: 4
Total Booked Tickets: 23

===== Cinema Menu =====
1. Book Tickets
2. Cancel Tickets
3. Show Available Seats
4. Show Total Booked
5. Exit
Enter your choice: 5
```

```
5. Exit
Enter your choice: 4
Total Booked Tickets: 23

===== Cinema Menu =====
1. Book Tickets
2. Cancel Tickets
3. Show Available Seats
4. Show Total Booked
5. Exit
Enter your choice: 5

Movie: 3 idiots
Available Seats: 102
Total Booked Tickets: 23
Thank You!
PS C:\Users\diyat\OneDrive\Documents\Dia study\java lab>
```

CONCLUSION

This practical helped me understand the use of classes, constructors, private, final, and static variables in Java. It also demonstrated how to manage cinema ticket booking and cancellation while updating seat availability and total booked tickets.

3. (For Advanced Learners – home) Parking lot (ParkingLot.java)

(a) Create class ParkingLot with private int twoWheelers and fourWheelers, private final int twoCap and fourCap, and private static long revenue=0.

(b) park(String type): for “two”, if twoWheelers < twoCap increment and add 20 to revenue; for “four”, if fourWheelers < fourCap increment and add 40; otherwise print “Full” and reject.

(c) leave(String type): decrement the matching count but never below 0.

(d) In main: simulate a sequence of park/leave events, printing each outcome, then print final occupancy and revenue.

Expected output: parking into a full section is refused; revenue = 20×(two parked)+ 40×(four parked).

CODE

```
import java.util.Scanner;
class ParkingLot {
    private int twoWheelers;
    private int fourWheelers;
    private final int twoCap;
    private final int fourCap;
    private static long revenue = 0;
    ParkingLot(int twoCap, int fourCap) {
        this.twoCap = twoCap;
        this.fourCap = fourCap;
        twoWheelers = 0;
        fourWheelers = 0;
    }
    void park(String type) {
        if (type.equals("two")) {
            if (twoWheelers < twoCap) {
                twoWheelers++;
```

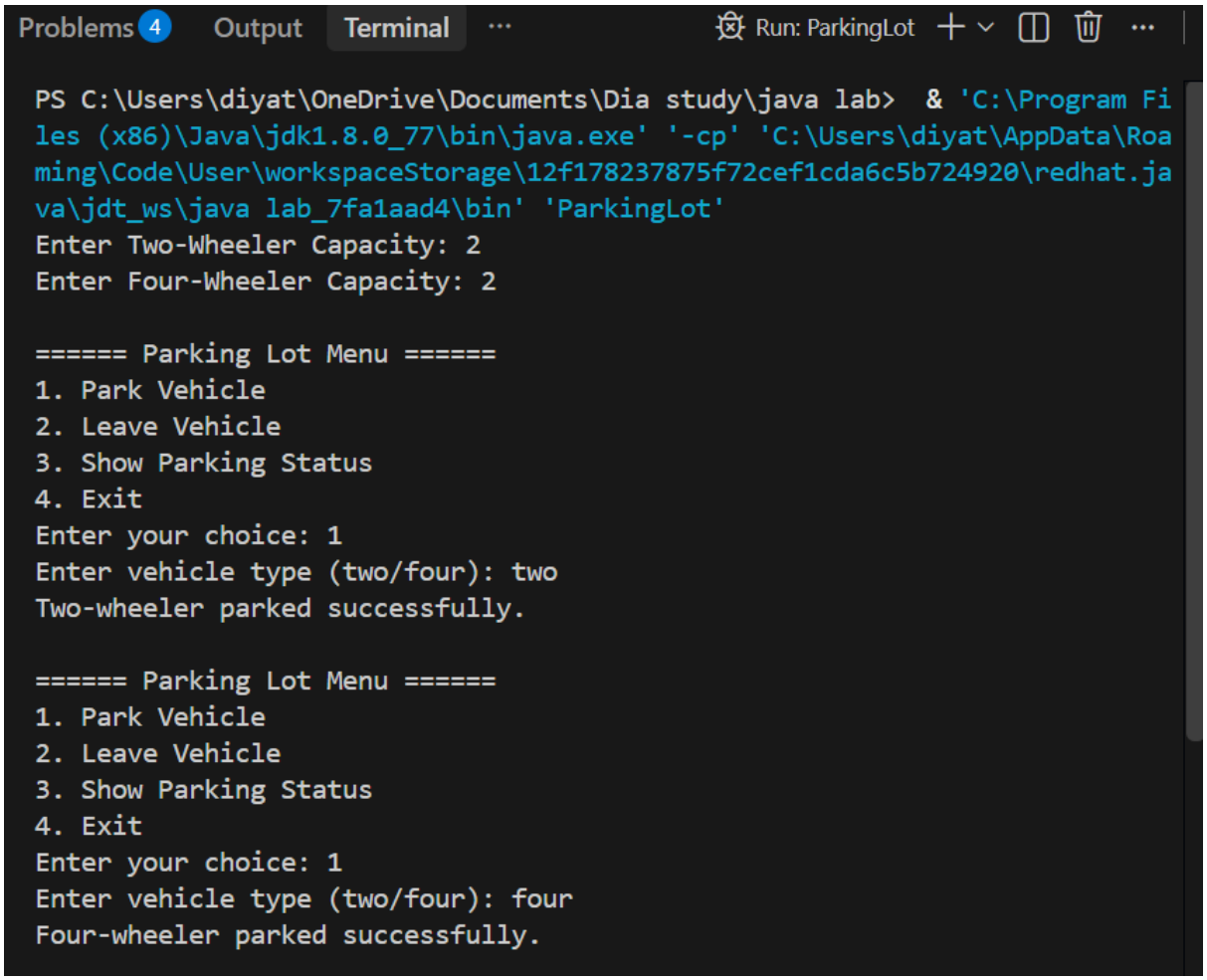
```
            revenue = revenue + 20;
            System.out.println("Two-wheeler parked successfully.");
        } else {
            System.out.println("Two-wheeler section is Full.");
        }
    }
    else if (type.equals("four")) {
        if (fourWheelers < fourCap) {
            fourWheelers++;
            revenue = revenue + 40;
            System.out.println("Four-wheeler parked successfully.");
        } else {
            System.out.println("Four-wheeler section is Full.");
        }
    }
    else {
        System.out.println("Invalid vehicle type.");
    }
}
void leave(String type) {
    if (type.equals("two")) {
        if (twoWheelers > 0) {
            twoWheelers--;
            System.out.println("Two-wheeler left.");
        } else {
            System.out.println("No two-wheelers are parked.");
        }
    }
    else if (type.equals("four")) {
        if (fourWheelers > 0) {
            fourWheelers--;
            System.out.println("Four-wheeler left.");
        } else {
            System.out.println("No four-wheelers are parked.");
        }
    }
    else {
        System.out.println("Invalid vehicle type.");
    }
}
void display() {
    System.out.println("\nCurrent Parking Status");
    System.out.println("Two-Wheelers : " + twoWheelers);
    System.out.println("Four-Wheelers: " + fourWheelers);
    System.out.println("Revenue      : Rs. " + revenue);
}
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter Two-Wheeler Capacity: ");
        int two = sc.nextInt();
        System.out.print("Enter Four-Wheeler Capacity: ");
        int four = sc.nextInt();
        ParkingLot p = new ParkingLot(two, four);

        while (true) {
            System.out.println("\n====== Parking Lot Menu ======");
            System.out.println("1. Park Vehicle");
            System.out.println("2. Leave Vehicle");
            System.out.println("3. Show Parking Status");
            System.out.println("4. Exit");
            System.out.print("Enter your choice: ");
            int choice = sc.nextInt();

            if (choice == 1) {
                System.out.print("Enter vehicle type (two/four): ");
                String type = sc.next();
                p.park(type);
            }
            else if (choice == 2) {
                System.out.print("Enter vehicle type (two/four): ");
                String type = sc.next();
                p.leave(type);
            }
            else if (choice == 3) {
                p.display();
            }
            else if (choice == 4) {
                System.out.println("\nFinal Parking Status");
                p.display();
                System.out.println("Thank You!");
                break;
            }
            else {
                System.out.println("Invalid Choice.");
            }
        }
        sc.close();
    }
}
```

OUTPUT



CONCLUSION

The Parking Lot program successfully implements object-oriented programming **concepts** in Java by using private data members, constructors, static variables, and methods. It manages the parking and leaving of two-wheelers and four-wheelers while enforcing capacity limits. The program also calculates the total revenue based on successfully parked vehicles and displays the final parking status. Thus, the program provides an effective simulation of a basic parking lot management system.

Key Questions / Analysis / Interpretation to be evaluated during/after Implementation

1. What is encapsulation, and how do private fields with public getters enforce it?

Answer: Encapsulation means wrapping data and methods together inside a class and restricting direct access to the data.

Fields are declared private, so they cannot be accessed directly from outside the class. Public getters are used to safely read the values.

A setter is not provided when we don't want outside code to change the value.

Example:

```
private int temperature;

public int getTemperature() {
   return temperature;
}
```

Here, temperature cannot be directly changed from outside the class. This provides data security and control.

2. Why is accountNumber declared final?

Answer: accountNumber is declared final because an account number should not change after it is assigned to an account.

For example:

private final int accountNumber;

Once a value is assigned to a final variable, it cannot be changed. This prevents accidental modification of the account number.

3. What is the difference between a static field and an instance field when generating ids?

Answer: Static field: One copy is shared by all objects of the class. It is useful for generating unique IDs because every object can use the same counter.

Instance field: Each object has its own separate copy. It cannot maintain one common ID counter for all objects.

Example:

```
private static int nextId = 1;
private int id;

id = nextId++;
```

Here, nextId is static, so all objects share the same counter. Each new object gets a different ID.